

vide — session launcher (shell)

<code>vide -n NAME</code>	Start or reattach the tmux session vide_NAME
<code>vide -q NAME</code>	Terminate session NAME
<code>vide -l</code>	List active vide sessions (Attached/Detached)
<code>vide -d DIR</code>	Set working directory (default: cwd)
<code>vide -h</code>	Show help

Top pane: Vim (75%). Bottom pane: console (25%). If a file under DIR matches NAME it opens directly, else main or the first .py/.tex.

tmux — prefix is `Ctrl-a`

<code>Ctrl-a</code>	Prefix key (replaces <code>Ctrl-b</code>)
<code>Ctrl-a Ctrl-a</code>	Send a literal <code>Ctrl-a</code> through
<code>Ctrl-a X</code>	Kill current session (confirm y/n)
mouse	Enabled: click panes, drag borders, scroll
<code>v</code> / <code>y</code>	copy-mode-vi: begin selection / yank & cancel

Panes collapse when unfocused; Tagbar & file explorer expand on entry.

IDE core — `,c` maps (all buffers)

<code>,cp</code>	Open / reset IDE layout (Tagbar + Editor + Files)
<code>,cf</code>	Find all occurrences of word under cursor
<code>,cg</code>	Jump to file+line of occurrence under cursor
<code>,cr</code>	Write find-list edits back to source files
<code>,cu</code>	Refresh the file explorer
<code>,cm</code>	Fetch OpenRouter free models, pick a default
<code>,f</code>	Open file path under cursor in editor window

Tagbar: double-click or `↩` jumps to the definition.

Find & replace workflow

- `,cf` — grep word under cursor across .py .tex .cpp .h .hpp under cwd; results shown as an editable list.
- Edit list lines in place to change the text.
- `,cr` — write modified lines back to their files.
- `,cg` — jump to the file/line of the row under cursor.

Screen layout

```
vide_NAME / window "code"
+-----+-----+-----+
| Tagbar| Editor | Files |
| symbol| main edit | netrw |
+-----+-----+-----+
| IPython console (slime)
| or plain bash
+-----+-----+-----+
```

Top = pane 0 (75%), console = pane 1 (25%). Tagbar & Files shrink to 2 cols when unfocused, expand on entry. `,cp` rebuilds the layout anytime.

First run

<code>./install.sh</code>	Deploy vimrc, tmux.conf, plugins, ~/bin/vide
<code>./install.sh -l</code>	Also install LaTeX system deps (texlive, okular)
<code>./install.sh -u</code>	Uninstall, restore .orig backups
<code>vide -n proj</code>	Launch / attach the IDE for session proj

Needs ~/.config/ai/openrouter.token for AI features. If cLaude CLI is logged in, AI routes to it by default.

Editor — clipboard & motion

<code>y</code> <code>p</code> <code>P</code>	normal & visual: auto-sync with the system clipboard via xclip
<code>↑</code> <code>↓</code>	Move by display line (gk/gj), wrap-aware
<code>Home</code> <code>End</code>	Display-line start / end

No xclip installed: y/p fall back to plain vim yank/paste.

Commenting (all filetypes)

<code>zc</code>	Comment selected / current lines
<code>zv</code>	Uncomment selected / current lines
<code>:Comment</code>	Comment a : '<', '>' range
<code>:Uncomment</code>	Uncomment a range
<code>\$</code>	TeX: reformat indent of selected comment block

Git grep

<code>\g</code>	Git grep whole-word for word under cursor
<code>\t</code>	Return to previous location after a grep jump
<code>:GG pat</code>	Git grep arbitrary pattern
<code>:GGw pat</code>	Git grep whole-word
<code>:GGi pat</code>	Git grep case-insensitive

Buffers & misc

<code>:Bdi</code>	Close all hidden (non-visible) buffers
<code>:DirDiff</code>	Directory diff (vim-dirdiff)
<code>\mm</code>	visual: add HTML anchors to Markdown headings
auto-save	All buffers written on focus lost
marks	Shown in sign column (vim-signature)
indent	4 spaces, expandtab; alternating indent guides

Python — IPython (vim-slime)	
<code>\ss</code>	Start IPython, Qt backend (--pylab qt)
<code>\as</code>	Start IPython, Agg backend (non-interactive)
<code>\hh</code>	n: send current line x: send selection
<code>\aa</code>	As <code>\hh</code> , then advance cursor one line
<code>\rr</code>	Run whole script (%run)
<code>\RR</code>	Run whole script and report run time
<code>\dd</code>	Debug current script (%run -d)
<code>\qq</code>	Exit debug mode / IPython (exit)
<code>\QQ</code>	Restart IPython
<code>\hh</code> / <code>\aa</code> are blocked with a warning unless IPython sits at an empty In [n]: prompt.	

Python — code cells (# %%)	
<code>\cc</code>	Execute the current cell
<code>\vv</code>	Execute cell verbosely, jump to next cell
<code>\ll</code>	Clear console, execute cell, jump to next
<code>\LL</code>	Clear the IPython console
<code>\xx</code>	Close all Matplotlib figure windows
<code>[c</code> / <code>]c</code>	Jump to previous / next cell header
<code>F9</code>	Insert cell header above + insert mode
<code>F10</code>	Insert cell header below + insert mode

Python — ALE lint / fix	
<code>Ctrl-k</code>	Previous lint error (wrapping)
<code>Ctrl-j</code>	Next lint error (wrapping)
<code>Ctrl-f</code>	Auto-fix file: black · isort · autoimport
<code>Ctrl-h</code> / <code>+BS</code>	Show hover documentation at cursor
<code>Ctrl-x</code>	Close the ALE hover / preview window
Linters: pylsp (flake8, mypy/dmypy) + jedi ls. black line length 90. Saving a .py updates its paired .ipynb (jupyterxtext).	

Shell buffers (filetype sh)	
<code>\ss</code>	Send echo "connect" to the console pane
<code>\hh</code>	n: send line x: send region to console
<code>\aa</code>	Send line / region, then advance one line

LaTeX (vimtex)	
<code>§</code>	Reformat indent of selected comment block
<code>F9</code>	insert mode: trigger LaTeX completion
<code>\ll</code>	Toggle latexmk continuous compilation
<code>\lv</code>	Forward search: view PDF in Okular
<code>\lk</code> / <code>\lc</code>	Stop compiler / clean aux files
<code>\le</code> / <code>\lt</code>	Show errors / toggle table of contents
Compiler: latexmk (bibtex, syntex, shell-escape, nonstop). Okular inverse search → vim --servername vim. All folds open. ALE off per-buffer.	

AI — vim-ai commands	
<code>:AI</code> prompt	Generate / complete text at cursor
<code>:AIEdit</code> instr	Edit selection / context in place
<code>:AIChat</code>	Open the AI chat scratch buffer
<code>:AIRedo</code>	Redo the last AI response
<code>,cm</code>	Pick a free OpenRouter model as default
Add a role after the command, e.g. <code>:AIChat /cc</code> or <code>:AIEdit /cc fix the bug</code> .	

AI — model roles	
<code>/cc</code> / <code>/claudecode</code>	Local claude CLI (Pro/Max billing)
<code>/free</code>	openrouter/free (OpenRouter default)
<code>/gemini</code>	google/gemini-3-flash-preview
<code>/gpt</code>	openai/gpt-4o
<code>/gpt5</code>	gpt-5.4
<code>/claude</code>	anthropic/claude-sonnet-4.6
<code>/opus</code>	anthropic/claude-opus-4.6
If claude is logged in at install, plain <code>:AI*</code> routes to <code>/cc</code> and OpenRouter becomes <code>/free</code> .	

AI — prompt roles (on selection)	
<code>/refine</code>	Rewrite text for clarity and flow
<code>/translate</code>	Translate text to English
<code>/professional</code>	Rewrite in a professional, expert tone
<code>/simplify</code>	Simplify language, drop jargon
<code>/clarity</code>	Make text direct and unambiguous
<code>/code</code>	Improve & format code (79-col)
<code>/refactor</code>	Refactor for clarity & maintainability
<code>/optimize</code>	Optimize for speed & memory
<code>/comment</code>	Add comments and docstrings
<code>/enhance-docstring</code>	Rewrite docstrings, Google format
<code>/enhance-comments</code>	Add & improve inline comments + docstrings
<code>/reformat-comments</code>	Reflow comments to 79 chars